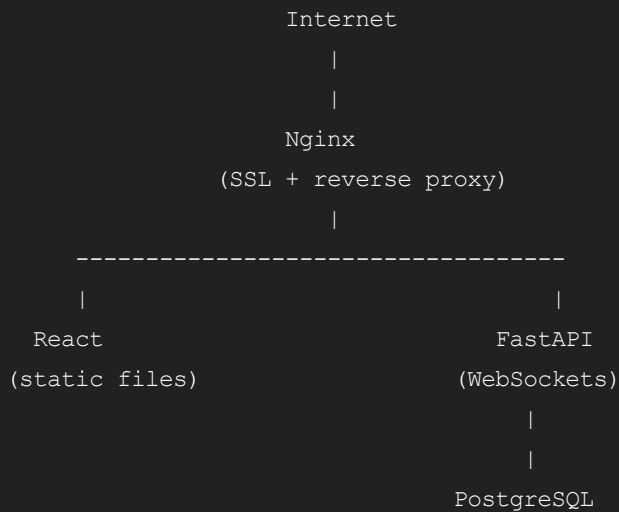


If you want to self-host with Docker + Nginx, that is actually a really good fit for your app. Your architecture is simple enough that you can run the whole thing on one VPS.

I would structure it like this:



With Docker:

```
docker-compose.yml

services:

  frontend
  |
  nginx container serving React build

  backend
  |
  FastAPI + uvicorn

  db
  |
  PostgreSQL

  nginx
  |
  reverse proxy + SSL
```

Recommended server

For your current game:

- 2 CPU
- 2-4GB RAM

- Ubuntu 24.04

Examples:

- Hetzner CX22
- DigitalOcean Basic Droplet
- Linode Nanode

Your workload is mostly idle except when people are playing.

Project structure

I would organise it like:

```
music-game/
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
├── frontend/
│   ├── Dockerfile
│   └── src/
├── nginx/
│   └── nginx.conf
├── docker-compose.yml
└── .env
```



Backend Dockerfile

backend/Dockerfile

```
<> dockerfile

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```



```
CMD [  
  "uvicorn",  
  "app.main:app",  
  "--host",  
  "0.0.0.0",  
  "--port",  
  "8000"  
]
```

Frontend Dockerfile

Since you use Vite:

frontend/Dockerfile

```
</> dockerfile   
  
FROM node:22-alpine AS build  
  
WORKDIR /app  
  
COPY package*.json .  
  
RUN npm install  
  
COPY . .  
  
RUN npm run build  
  
  
FROM nginx:alpine  
  
COPY --from=build /app/dist /usr/share/nginx/html
```

PostgreSQL

Your compose file handles this.

docker-compose.yml

```
</> YAML 
```

```
services:

  backend:
    build:
      context: ./backend

    env_file:
      - .env

    depends_on:
      - postgres

    restart: always

  frontend:
    build:
      context: ./frontend

    restart: always

  postgres:
    image: postgres:16

    environment:
      POSTGRES_USER: gameuser
      POSTGRES_PASSWORD: password
      POSTGRES_DB: musicgame

    volumes:
      - postgres_data:/var/lib/postgresql/data

    restart: always

  nginx:
    image: nginx:alpine

    ports:
      - "80:80"
      - "443:443"

    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./certbot/www:/var/www/certbot
      - ./certbot/conf:/etc/letsencrypt

    depends_on:
      - frontend
      - backend
```

```
restart: always
```

```
volumes:
```

```
postgres_data:
```

Nginx config

This is the important part because you use WebSockets.

```
nginx/nginx.conf
```

</> Nginx



```
events {}

http {

    server {

        listen 80;

        server_name yourdomain.com;


        location / {

            proxy_pass http://frontend;

        }


        location /api/ {

            proxy_pass http://backend:8000;

            proxy_set_header Host $host;

            proxy_set_header X-Real-IP $remote_addr;

        }


        location /rooms/ws/ {

            proxy_pass http://backend:8000;
```

```
        proxy_http_version 1.1;

        proxy_set_header Upgrade $http_upgrade;

        proxy_set_header Connection "upgrade";

        proxy_set_header Host $host;

    }

}
```

The websocket section is critical.

Without:

</> Nginx



```
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
```

your rooms will randomly fail.

Environment variables

Example:

.env

</> env



```
DATABASE_URL=postgresql+asyncpg://gameuser:password@postgres:5432/musicgame

JWT_SECRET=mysecret

SPOTIFY_CLIENT_ID=xxxx
SPOTIFY_CLIENT_SECRET=xxxx

FRONTEND_URL=https://yourdomain.com
```

Deployment steps

On your server:

</> Bash



```
git clone yourrepo
```

```
cd music-game
```

```
docker compose up -d --build
```

Check:

</> Bash



```
docker ps
```

Logs:

</> Bash



```
docker compose logs -f backend
```

SSL

Use Certbot:

Install:

</> Bash



```
sudo apt install certbot
```

Get certificate:

</> Bash



```
certbot certonly --standalone \  
-d yourdomain.com
```

Then mount:

```
/etc/letsencrypt
```



into nginx.

One important thing for your current backend

Your current:

Python



Run

```
self.game_rooms = {}  
self.connections = defaultdict(dict)
```

will work on this setup because you have **one FastAPI container**.

Do not scale horizontally yet.

If later you run:

```
backend-1  
backend-2  
backend-3
```



you will need Redis because players connected to different containers won't see each other.

For your current stage (Spotify multiplayer quiz with rooms), a single VPS running:

```
Nginx  
↓  
React container  
↓  
FastAPI container  
↓  
Postgres container
```



is exactly how I would deploy it. It will comfortably handle hundreds of concurrent players before you need a more complicated setup.



Sky

Ad ...

Power your working day
Stay productive with fast, stable internet.